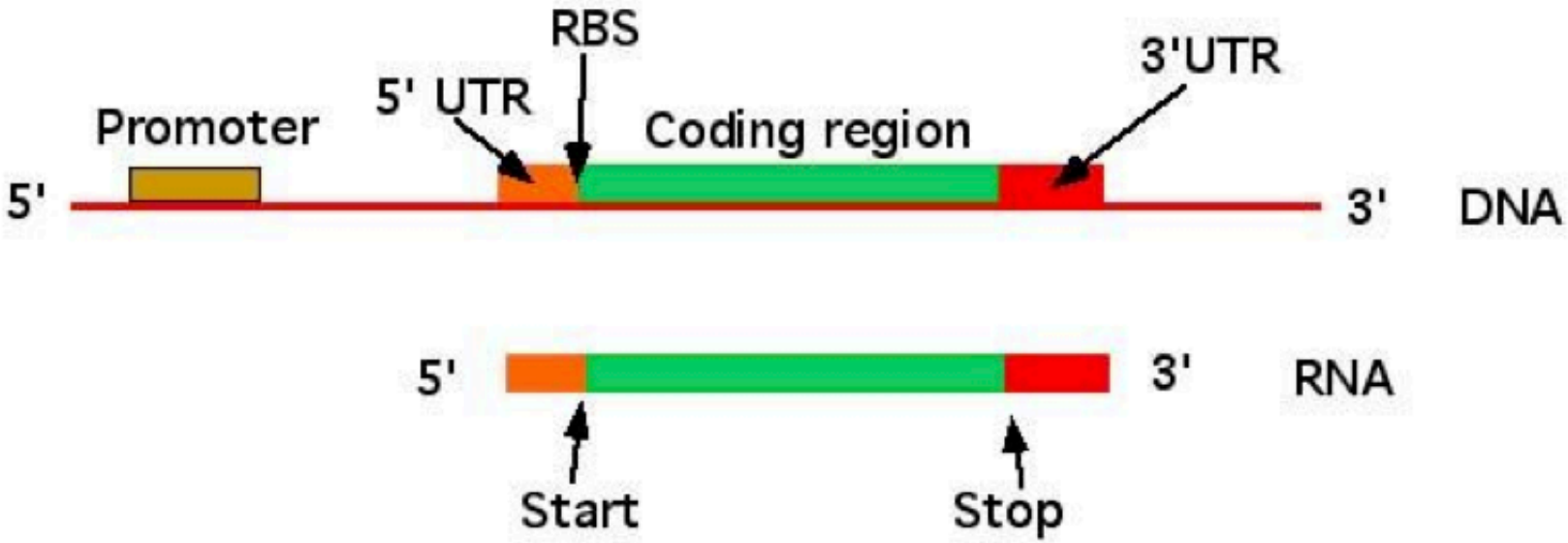
Gene annotation

The goal of this lecture will be to use detect automatically genes coding sequences from the information of the sequence of a prokaryotic genome, *Entorebacter clocae*.

A gene basic properties

The figure below summarizes the basic properties for a gene. In the following our goal is first to annotate the coding region (in green).



We will see how gene annotation can be done on the whole genome and compared to the real annotation in two steps:

1. we will first using this simple properties of a gene: it is a sequence between a start and a stop codon .
2. we will then use the degeneracy of the genetic code to help find regions which are more likely to be coding based on their compositionnal

bias.

Loading genomic sequence and annotations

We will be analysing the genome of *E. clocae* for which we have also a reference annotation. *E. clocae* is a clinically significant Gram-negative, facultatively-anaerobic, rod-shaped bacterium, more info (https://en.wikipedia.org/wiki/Enterobacter_cloacae)

Let's first load its genomic sequence and get the annotation in a dataframe.

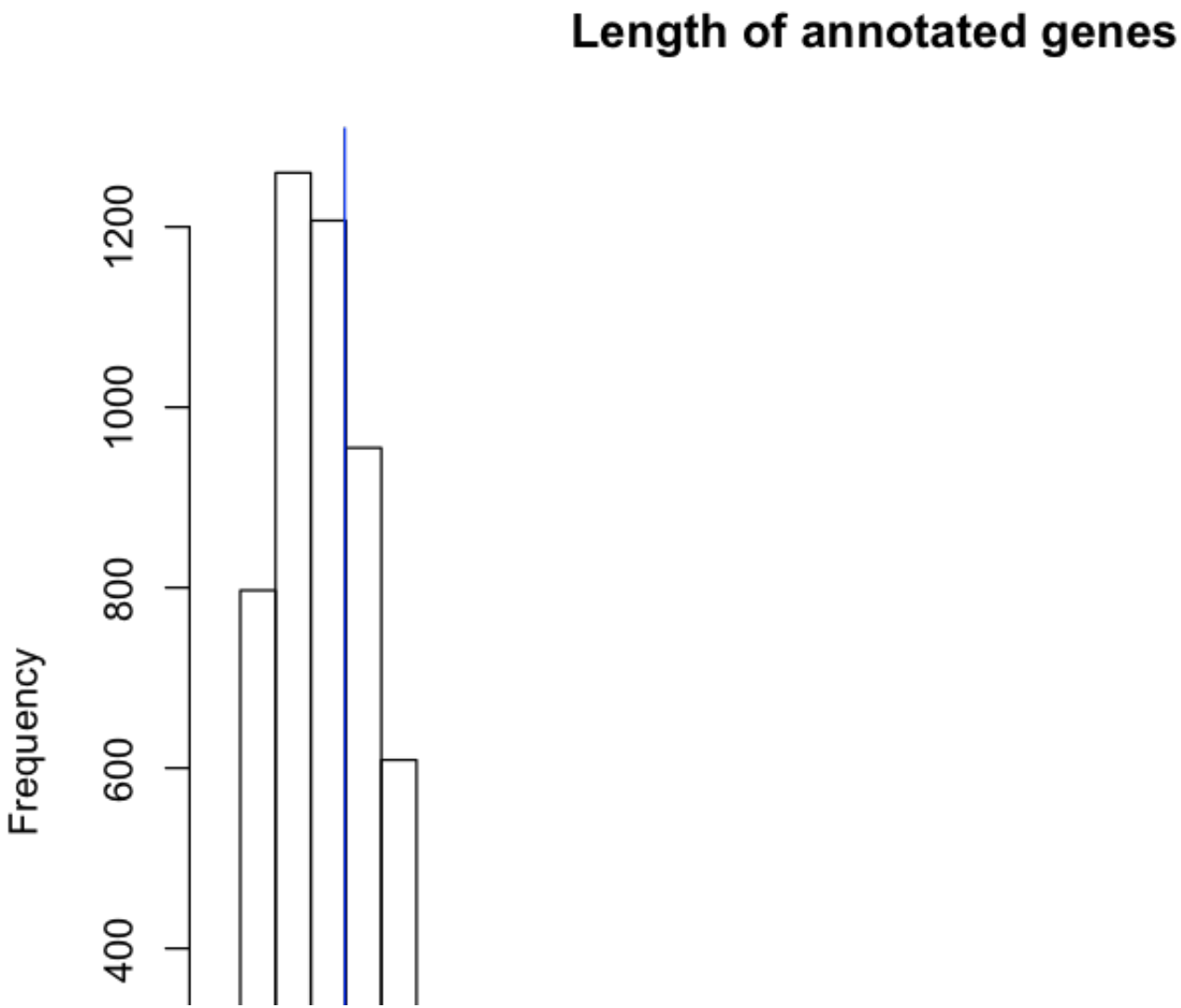
In [38]:
```
library(seqinr)

ecseq <- read.fasta(file = "http://www.lcqb.upmc.fr/hrichard/sequences/eclocae.genome.fasta")[[1]]

#Size of the genome
cat("Length of the E. clocae genome:", length(ecseq))

ecannot <- read.delim("http://www.lcqb.upmc.fr/hrichard/sequences/eclocae.annotation.tsv")
head(ecannot)
```

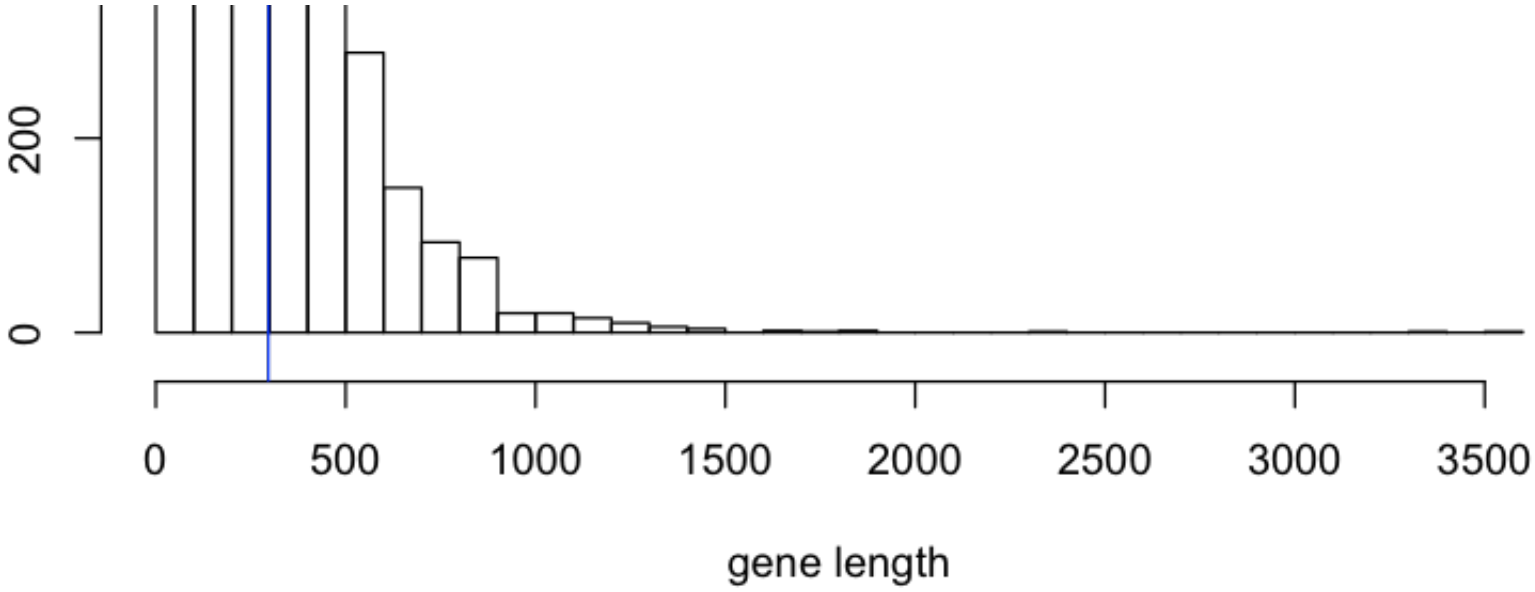
```
Length of the E. clocae genome: 5314581
```

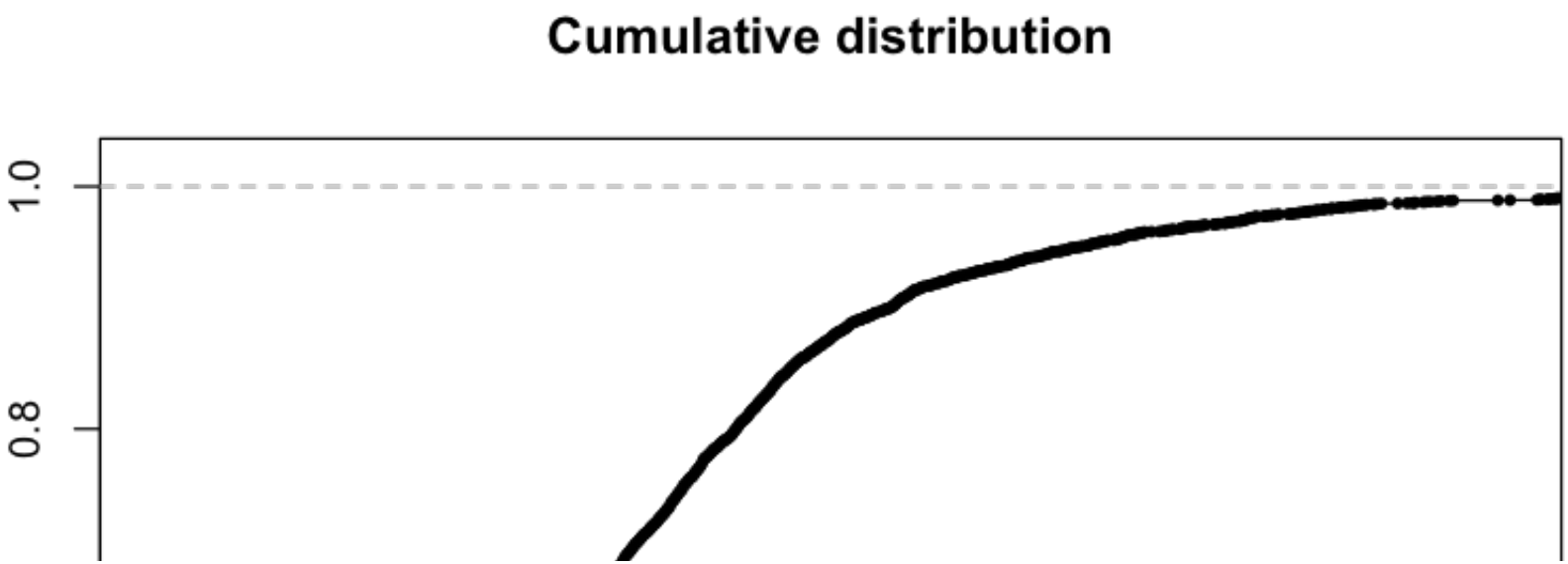
Replicon.Name	Replicon.Accession	Start	Stop	Strand	GeneID	Locus	Locus.tag	Protein.product	Length	COG.s.	Protein.name
chr	NC_014121.1	82	1410	+	9127753	dnaA	ECL_00001	YP_003610518.1	442	-	chromosomal replication initiator protein
chr	NC_014121.1	1415	2515	+	9122513	-	ECL_00002	YP_003610519.1	366	-	DNA polymerase III subunit beta
chr	NC_014121.1	2515	3588	+	9122514	-	ECL_00003	YP_003610520.1	357	-	recombination protein F
chr	NC_014121.1	3617	6028	+	9122515	gyrB	ECL_00004	YP_003610521.1	803	-	DNA gyrase subunit B
chr	NC_014121.1	6049	6996	-	9122516	-	ECL_00005	YP_003610522.1	315	-	LysR family transcriptional regulator
chr	NC_014121.1	7107	8132	+	9122517	-	ECL_00006	YP_003610523.1	341	-	alcohol dehydrogenase protein

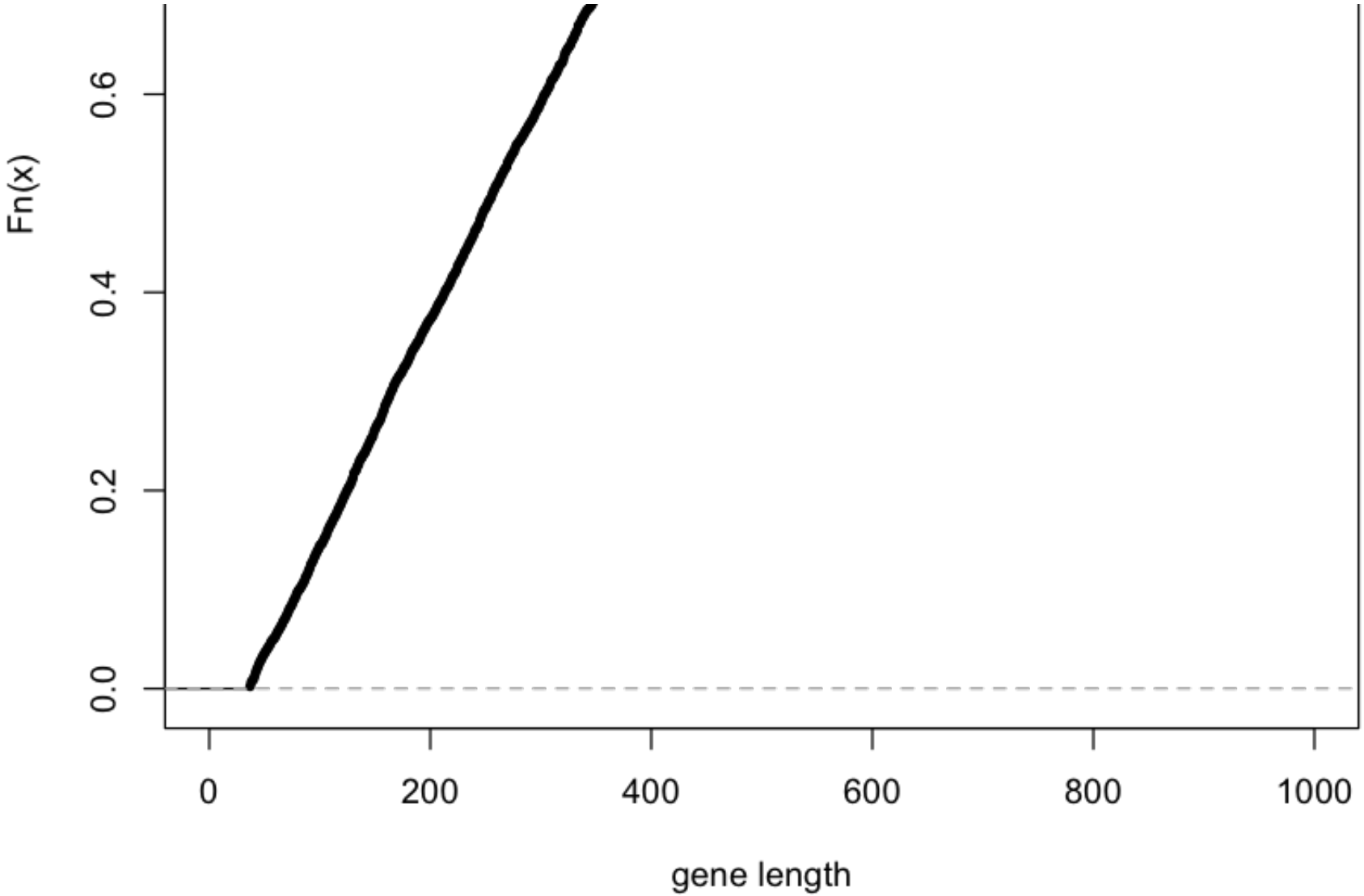
In [40]:
```

##Let's do an histogram of the annotated gene length

hist(ecannot$Length, br = 40, main = "Length of annotated genes", xlab = "gene length")
abline(v = mean(ecannot$Length), col = "blue")
```

```
plot(ecdf(ecannot$Length), xlim = c(0, 1000), cex = 0.5, main = "Cumulative distribution",
    xlab = "gene length")
```




200
0
0
500
1000
1500
2000
2500
3000
3500
gene length


Cumulative distribution
1.0
0.8



Annotating Open Reading Frames

An Open Reading Frame (ORF) is defined as a stretch of sequence such that:

1. It begins with a start codon (`ATG`, `GTG` or `TTG`)
2. It ends with a stop codon (`TAA`, `TAG` or `TGA`).
3. There are no in phase stop codon within the sequences.

The first way for people to annotate a genome was to mark, for each phase and on each strand, the stop and the start codon and detect the longest ORF at each position.

In [41]:
```
start = c("ATG", "GTG", "TTG", "atg", "gtg", "ttg")
stop = c("TAA", "TAG", "TGA", "taa", "tag", "tga")

##Do a function to return a list of positions with the phase
##for a given list of codons
findOcc <- function(ntseq, codonlist){
  occs = c()
  for (i in 1:(length(ntseq)-2)){
    ccod = paste(ntseq[i:(i+2)], collapse = "")
    if (ccod %in% codonlist){
      #cframe = (i %% 3 ) + 1
      occs = c(occs, i)
    }
  }
  posocc = data.frame(pos = occs, frame = (occs %% 3) + 1)
  return(posocc)
}

###Let's get the start and end codons on the first 10万 positions.

Startpos = findOcc(ecseq[1:100000], start)
Stoppos = findOcc(ecseq[1:100000], stop)
```

We can plot the results for a stretch of a chromosome, and compare with the annotation.

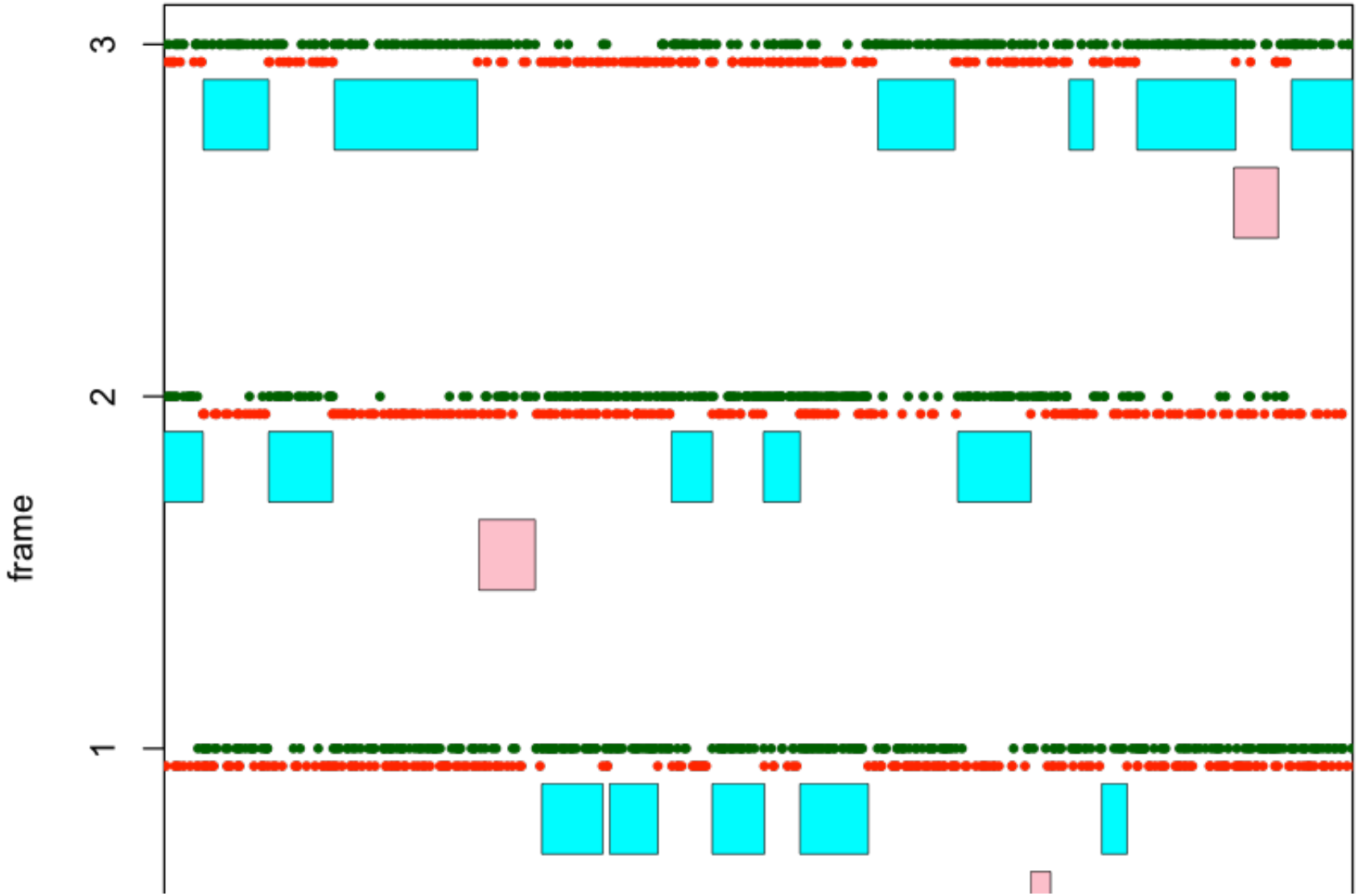
Let's first prepare a plotting function.

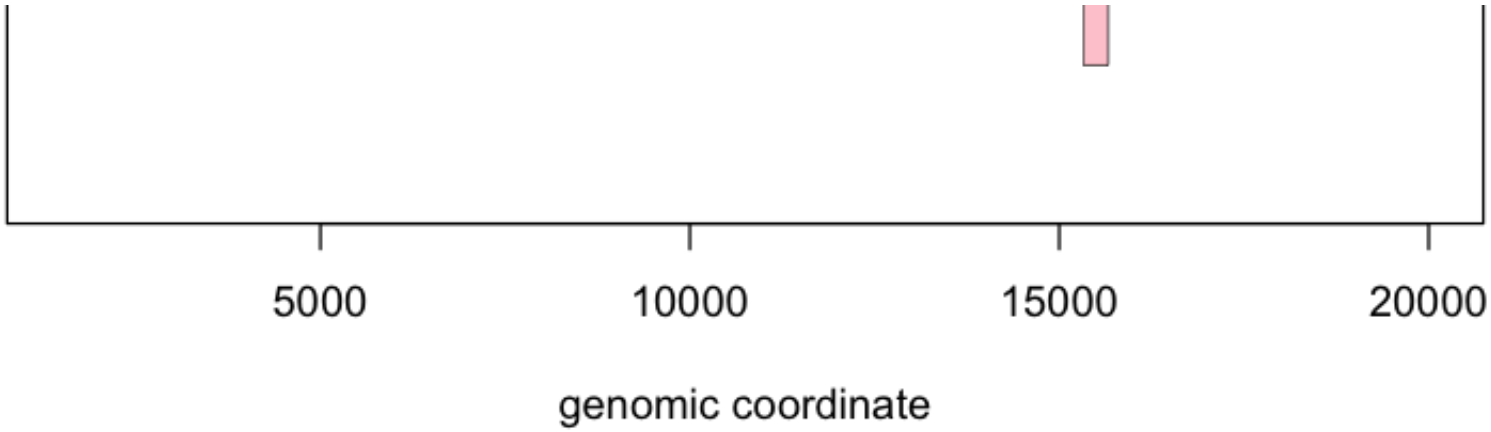
In [42]:
```
##Plots the annotation together with the on the positions from beginning to end
PlotPhaseAndAnnotation <- function(begin, end, startpos, stoppos, annot){
  plot(c(begin, end), c(0.2, 3), type =  "n",
       ylab = "frame", xlab= "genomic coordinate",
       yaxp = c(1,3, 2))

  points(startpos$pos, startpos$frame, col = "darkgreen",
         cex = 0.5, pch = 19)
  points(stoppos$pos, stoppos$frame-0.05, col = "red",
         cex = 0.5, pch = 19)

  genefw = subset(annot, Strand == "+" & Replicon.Name == "chr")
  genebw = subset(annot, Strand == "-" & Replicon.Name == "chr")
  ##Annotations
  rect(genefw$Start, ((genefw$Start) %% 3) + 0.7, genefw$Stop,
       ((genefw$Start) %% 3) + 0.9, col = "cyan", lwd = 0.5)

  rect(genebw$Start, ((genebw$Start) %% 3) + 0.65, genebw$Stop,
       ((genebw$Start) %% 3) + 0.45, col = "pink", lwd = 0.5)
}
```

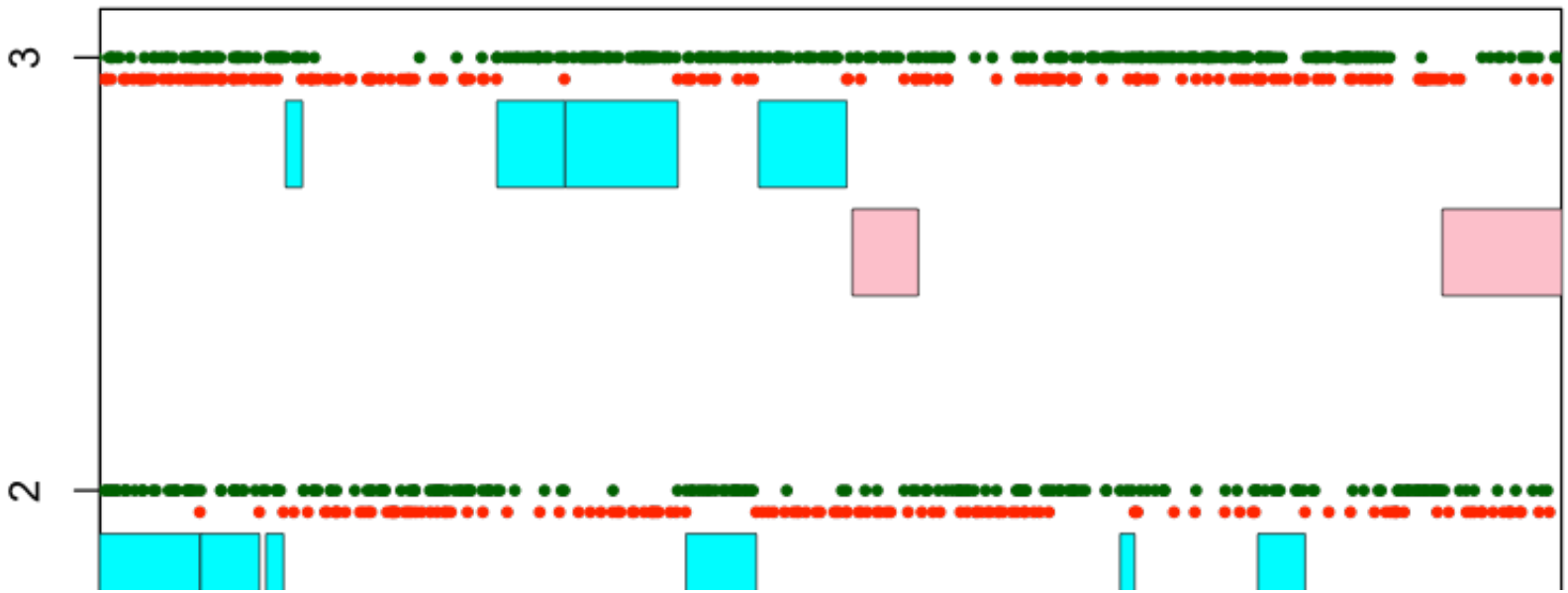
Let's plot some ranges of coordinates now.

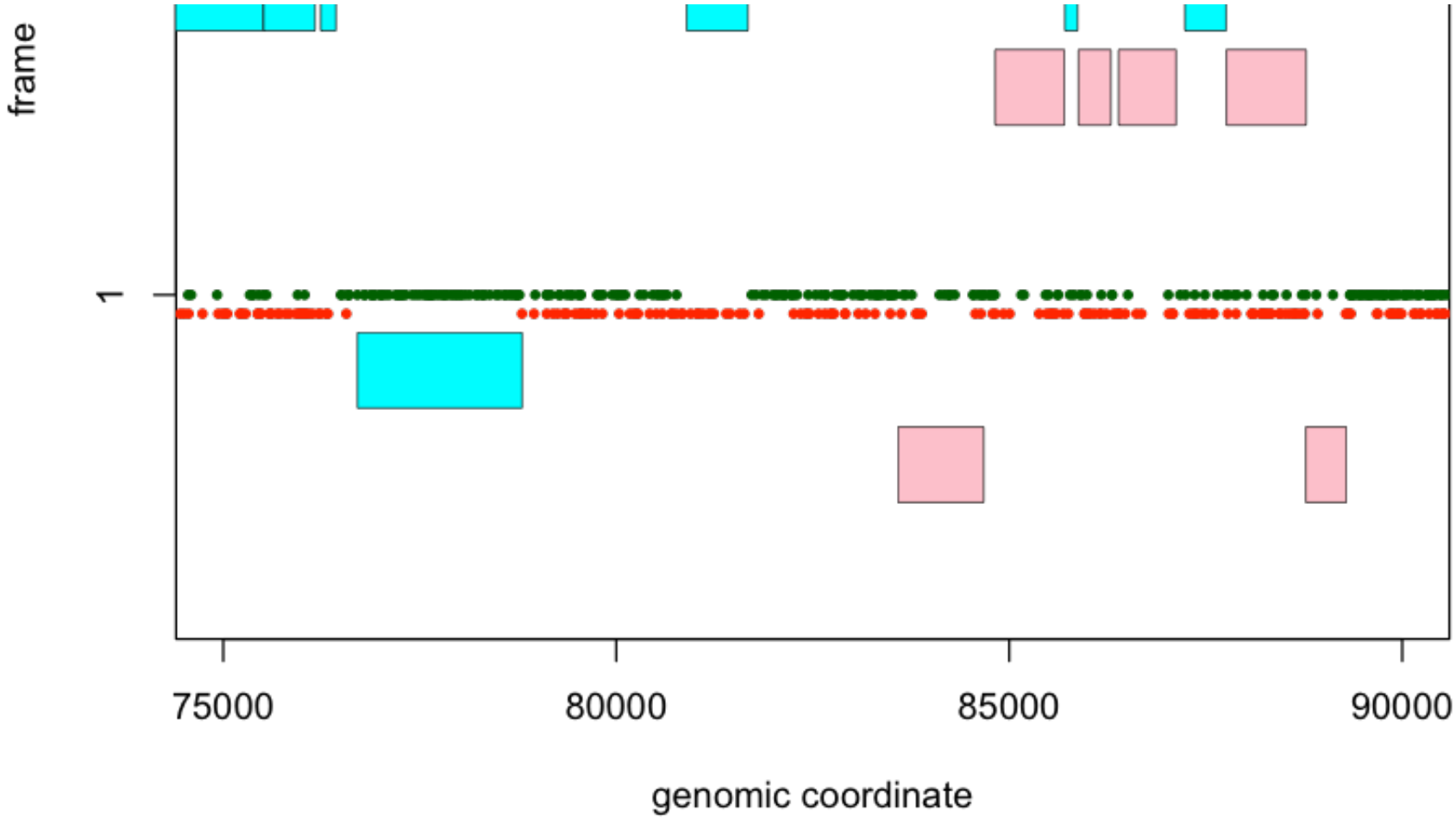
You can notice on the second plot that some long ORF occur with no genes on the forward strand (frame 2 in the middle)

In [43]:
```
PlotPhaseAndAnnotation(1500, 20000, Startpos, Stoppos, ecannot)

PlotPhaseAndAnnotation(75000, 90000, Startpos, Stoppos, ecannot)
```


frame
1
2
3


5000
10000
15000
20000
genomic coordinate


3
2

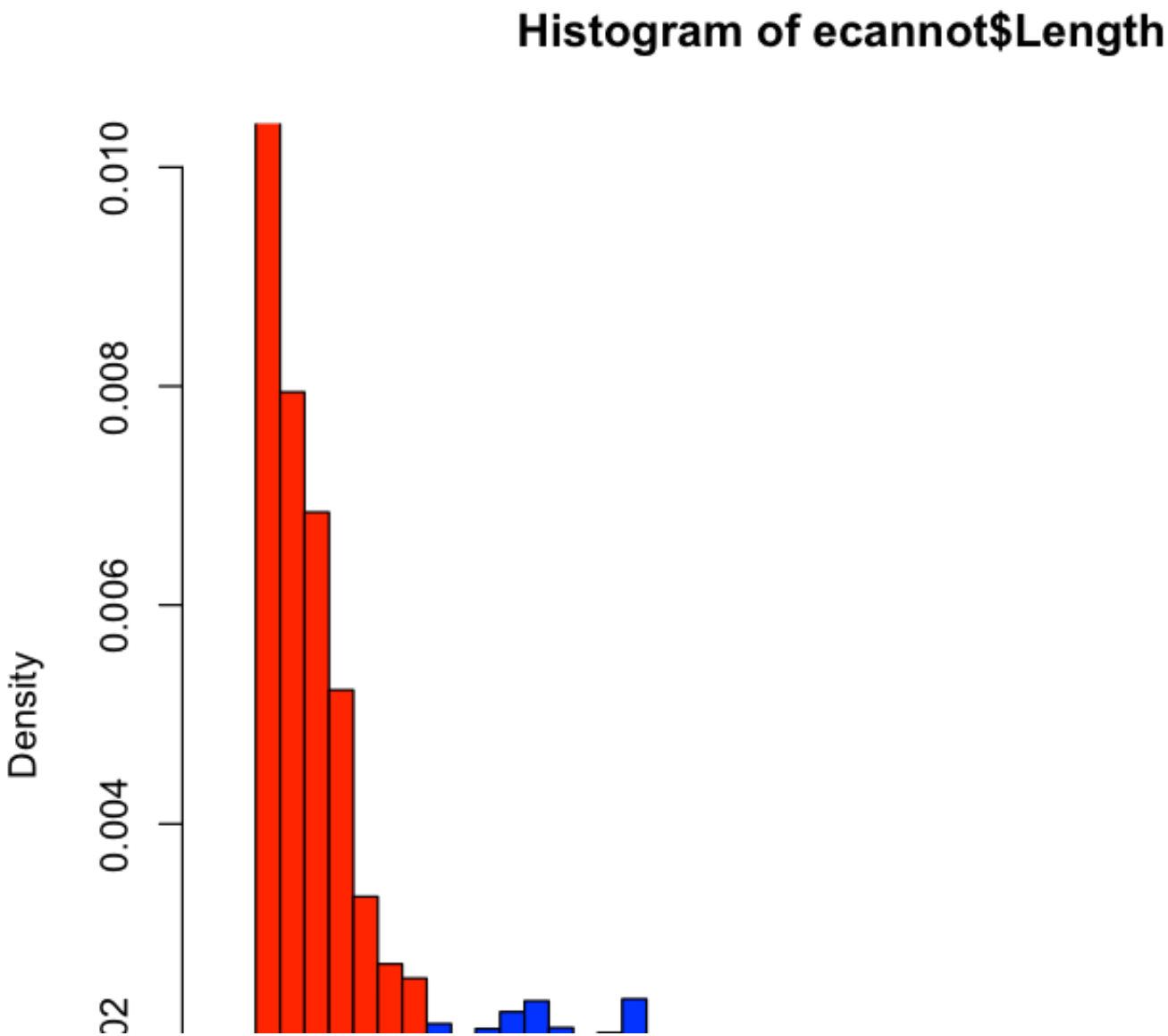


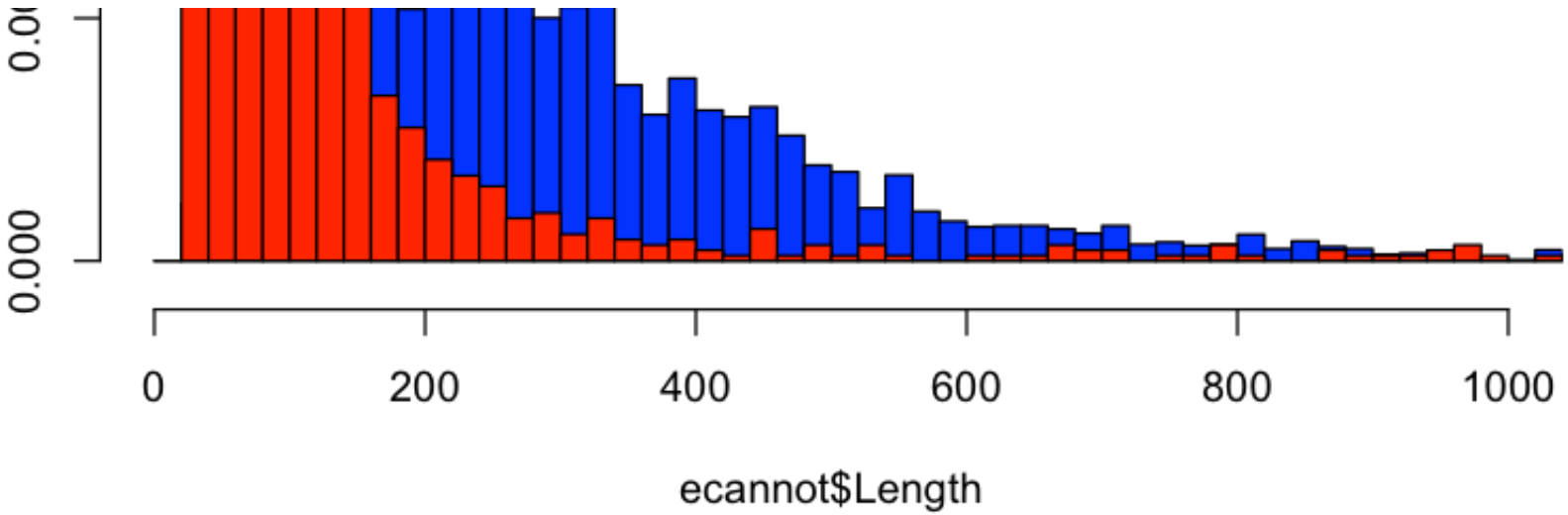
Let's detect all the ORFs from a collection of start and stop positions.

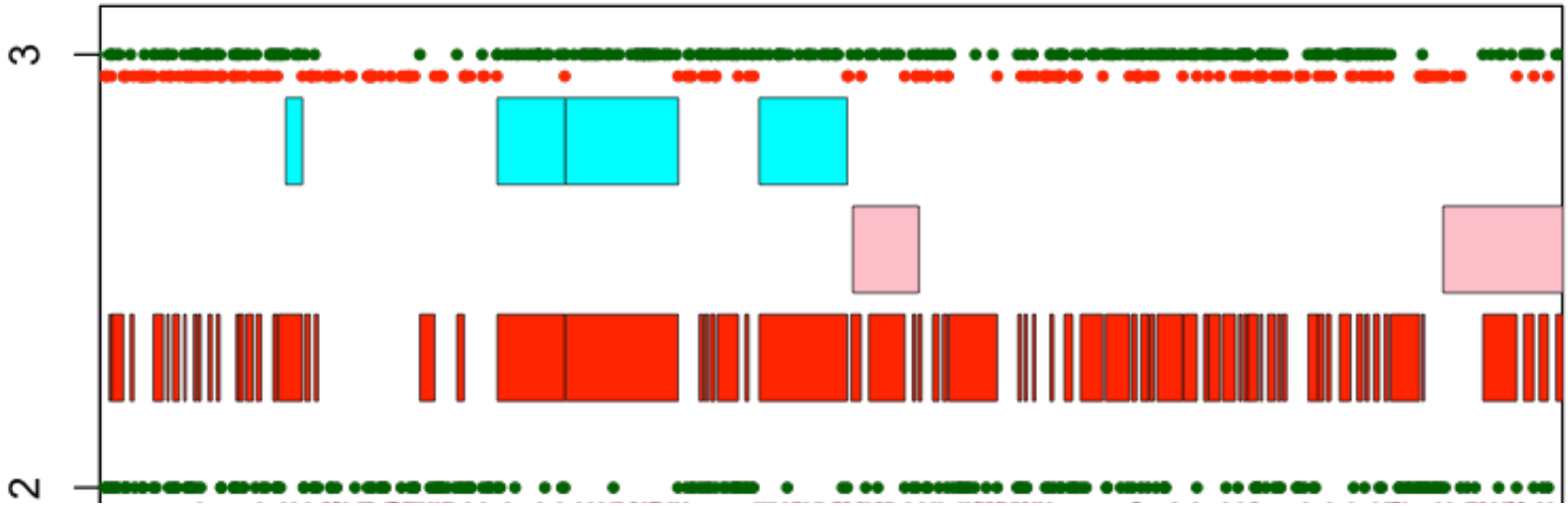
In [44]:
```
library(dplyr, quietly =TRUE)

allpos = rbind(data.frame(Startpos, type = 1), data.frame(Stoppos, type = -1))
allpos = allpos %>% arrange(pos) %>% group_by(frame) %>%
          mutate(stopnum = cumsum(lag(as.integer(type == -1), default = 0)))

by_frame = allpos %>% group_by(frame, stopnum)
ORFpos = summarise(by_frame, start = min(pos), end = max(pos),
                   tstart = type[which.min(pos)],
                   tend = type[which.max(pos)],
                   len = end - start + 1)
ORFpos = ORFpos %>% filter(len > 20)
```

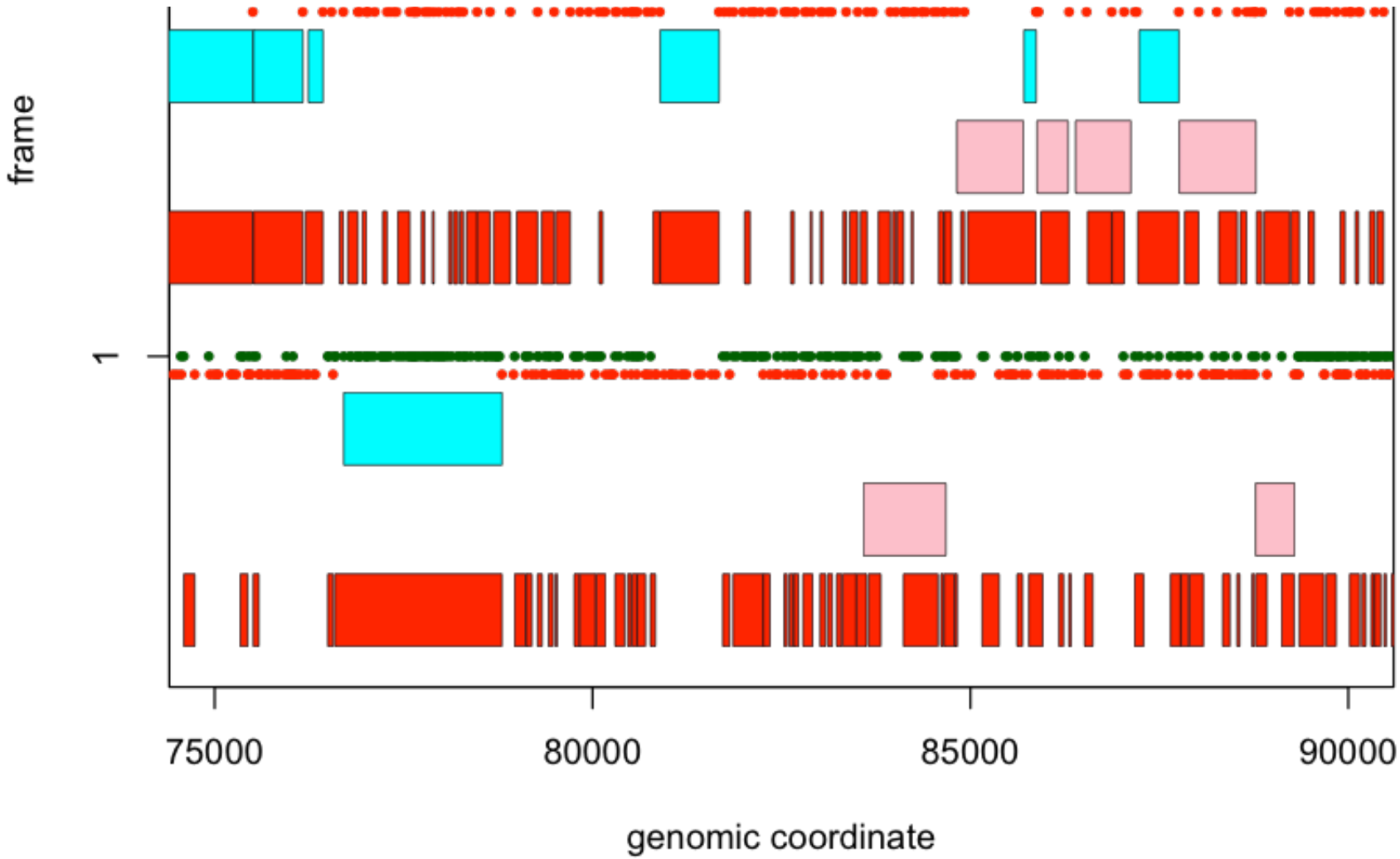
We can now look at the histogram of the ORF length and compare it to the histogram of the real genes in order to decide at which value it is unlikely that a ORF was due to chance.

In [45]:
```
## Plot the histogram of lengths
##

hist(ecannot$Length, br = seq(0, 100000, by = 20),
     xlim = c(0,1000), ylim = c(0, 0.01), col = "blue", freq = FALSE)
hist(ORFpos$len, br = seq(0, 100000, by = 20), add = TRUE, freq = FALSE, col = "red", alpha= 0.2)

##We also compare the ORF with the annotation on a stretch


PlotPhaseAndAnnotation(75000, 90000, Startpos, Stoppos, ecannot)

rect( ORFpos$start, ORFpos$frame - 0.6, ORFpos$end, ORFpos$frame - 0.8,
      col = "red", lwd = 0.5)
```


Histogram of ecannot$Length
Density
0.002
0.004
0.006
0.008
0.010


0.000
0
200
400
600
800
1000
ecannot$Length


3
2



A classical threshold is usually set at 100 nt. But in this case we miss some of the shorter genes. Those are called False Negative predictions.

In [46]:
```
table(ecannot$Length < 100)
```

```
FALSE  TRUE 
 4739   779 
```

With this threshold we would miss 779 genes.

Statistical properties of coding regions.

In order to detect more genes we will assess the coding potential of the regions by comparing its probability under two different models:

- **The codons model:** in this model the probability of a sequence is the product of the probabilities of all the codons which are occurring.
- **The independant nucleotides models:** in this model the probability of a sequence is the product of the frequencies of its nucleotides.

We can use a sliding window of a given size and compare in each window compute the log ratio of the probabilities.

The probabilities of all codons are precomputed and given in the beginning. We write a function to compute the log ratio of probabilities for a given window.

In [51]:
```
pcodons = c(aaa = 0.0287,aac = 0.02533,aag = 0.01372,aat = 0.01221,aca = 0.00505,
            acc = 0.02957,acg = 0.01735,act = 0.00475,aga = 0.00197,agc = 0.01977,
            agg = 0.00173,agt = 0.00577,ata = 0.00335,atc = 0.02926,atg = 0.02716,
            att = 0.02405,caa = 0.00792,cac = 0.01197,cag = 0.0355,cat = 0.01034,
            cca = 0.00647,ccc = 0.00685,ccg = 0.02485,cct = 0.00671,cga = 0.00278,
            cgc = 0.02546,cgg = 0.00639,cgt = 0.01872,cta = 0.00209,ctc = 0.01366,
            ctg = 0.06566,ctt = 0.01106,gaa = 0.03175,gac = 0.0245,gag = 0.02462,
            gat = 0.02809,gca = 0.01557,gcc = 0.03395,gcg = 0.03721,gct = 0.01111,
            gga = 0.00671,ggc = 0.03416,ggg = 0.01381,ggt = 0.01989,gta = 0.00781,
            gtc = 0.01699,gtg = 0.03232,gtt = 0.01433,taa = 3e-05,tac = 0.01453,
            tag = 3e-05,tat = 0.01333,tca = 0.00615,tcc = 0.01153,tcg = 0.00926,
            tct = 0.00704,tga = 4e-05,tgc = 0.00663,tgg = 0.01568,tgt = 0.00388,
            tta = 0.00701,ttg = 0.01932,ttg = 0.00752,ttt = 0.01897)
```

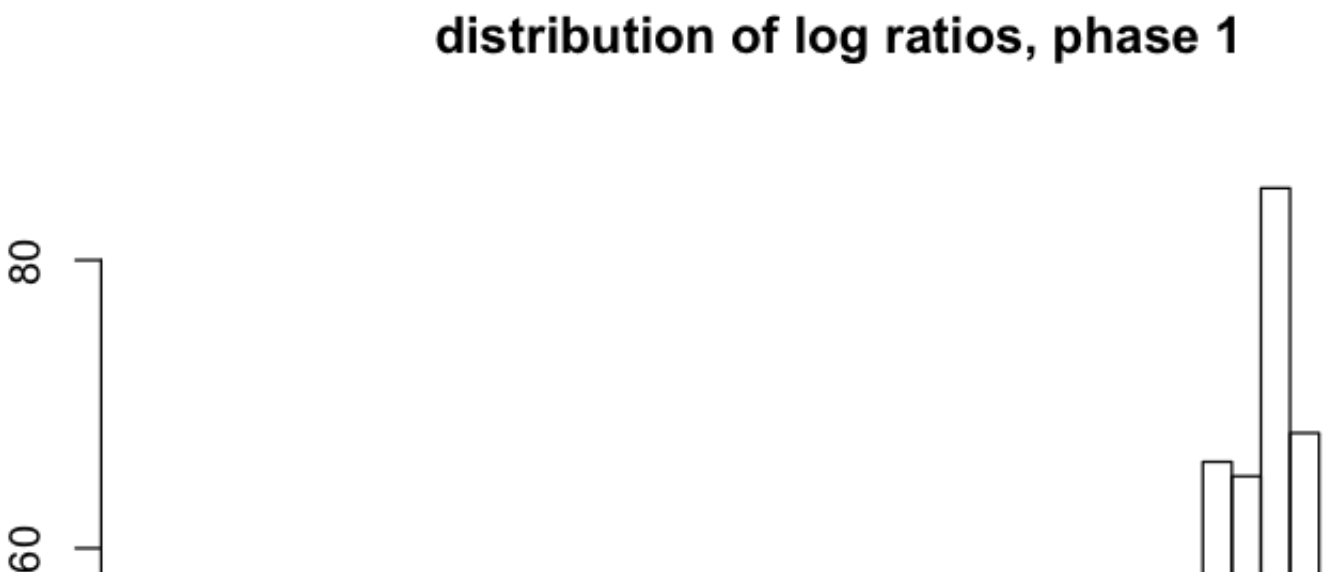
```
            tta = 0.00701,ttc = 0.01932,ttg = 0.00752,ttt = 0.01897)

ntproba = seqinr::count(ecseq[1:10000], wordsize = 1, freq =TRUE)

##The function

CodingScore <- function(cseq, pcodon, pnt){
  lpcodon = log(pcodon)
  lpnt = log(pnt)
  ntlogproba = 0
  codonlogproba = 0
  for (i in seq(1,(length(cseq)-2), by = 3)){
    ccodon = cseq[i:(i+2)]
    cc = paste(ccodon, collapse = "")
    ntlogproba = ntlogproba + lpnt[[ccodon[1]]] + lpnt[[ccodon[2]]] + lpnt[[ccodon[3]]]
    codonlogproba = codonlogproba + lpcodon[[cc]]
  }
  logoddsratio = codonlogproba - ntlogproba
  return(logoddsratio)
}

##The window function, we just compute it every 4 codons.
CodingWindow <- function(cseq, start, end, pcodon, pnt, windowsize = 66, smsize =5){
   realend = end - windowsize + 1
   xcoords = seq(start, realend, by = 12)
   scores = sapply(xcoords,
                   function(x) {
                     to = x+windowsize-1
                     CodingScore(cseq[x:to], pcodon, pnt)})
   res = data.frame(pos = xcoords, logp = scores)
   res$winlogp = 0
   for (x in (smsize+1):(nrow(res)-smsize)){
       res$winlogp[x] = mean(res$logp[(x-smsize):(x+smsize)])
   }
   res$scalelogp = res$winlogp - min(res$winlogp)
   res$scalelogp = res$scalelogp / max(res$scalelogp)
```

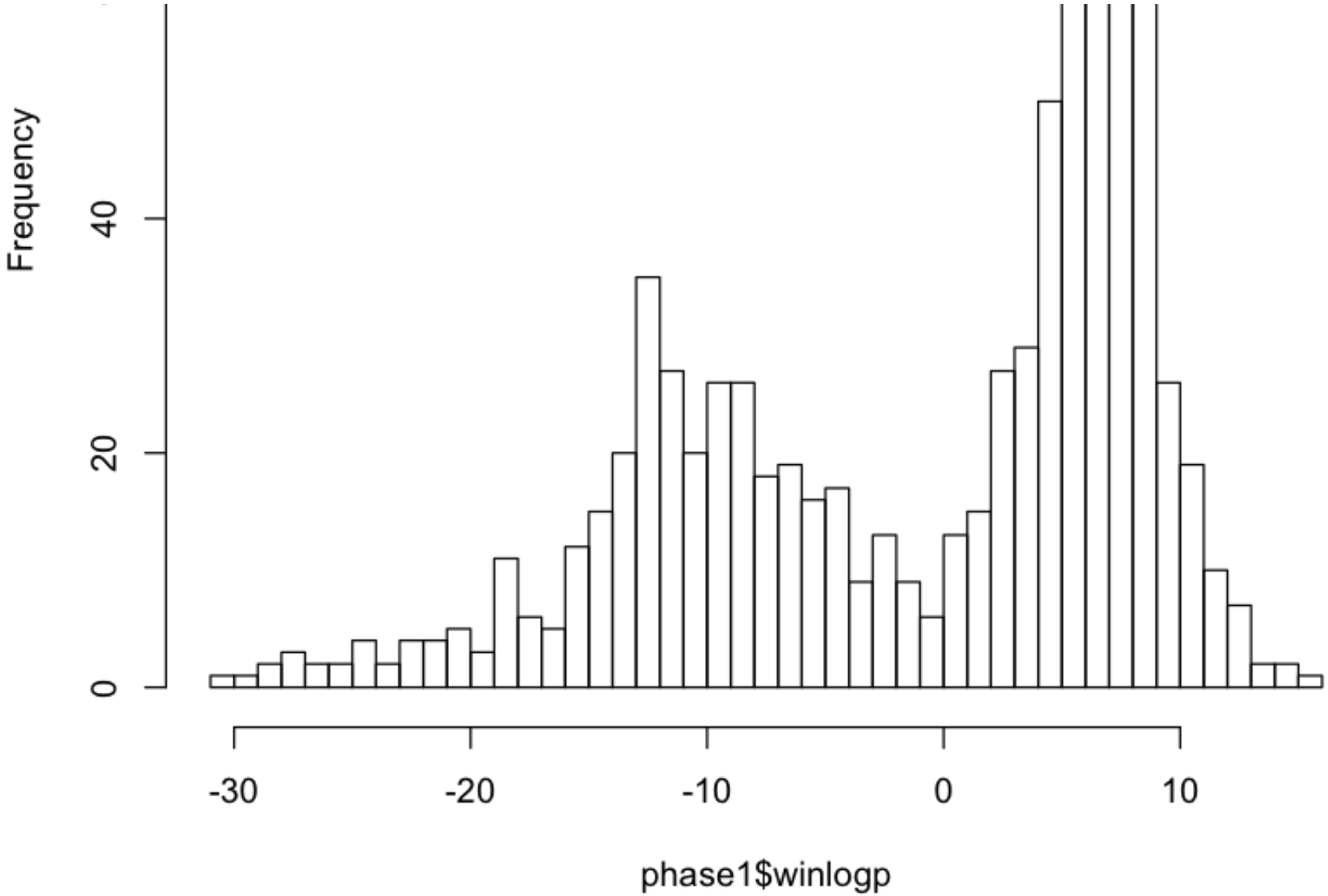
```
  return(res)
}
```

Let's do this analysis for all the three phases in one region.

In [55]:
```
phase1 = CodingWindow(ecseq, 70002, 80000, pcodons, ntproba, smsize = 1)
phase2 = CodingWindow(ecseq, 70003, 80000, pcodons, ntproba, smsize = 1)
phase3 = CodingWindow(ecseq, 70001, 80000, pcodons, ntproba, smsize = 1)
```

In [56]:
```
##Check the distribution of the logratio of windows

hist(phase1$winlogp, br = 40, main = "distribution of log ratios, phase 1")
```

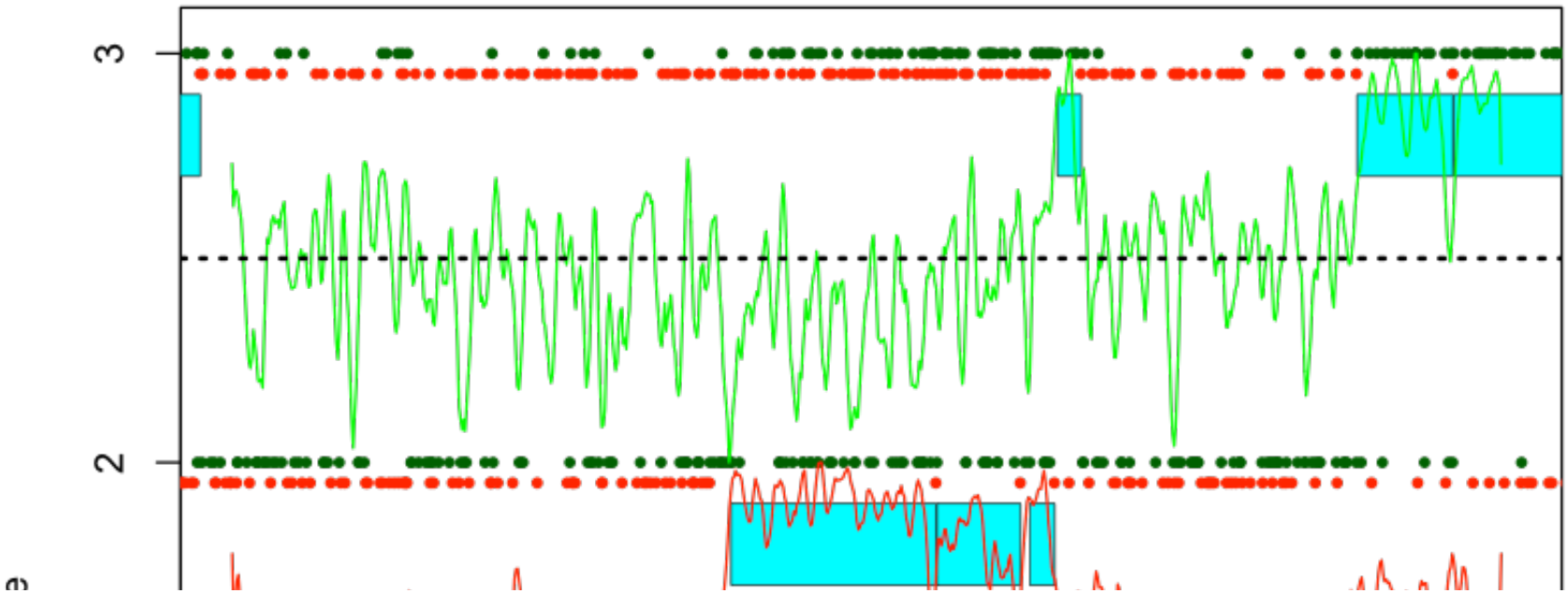


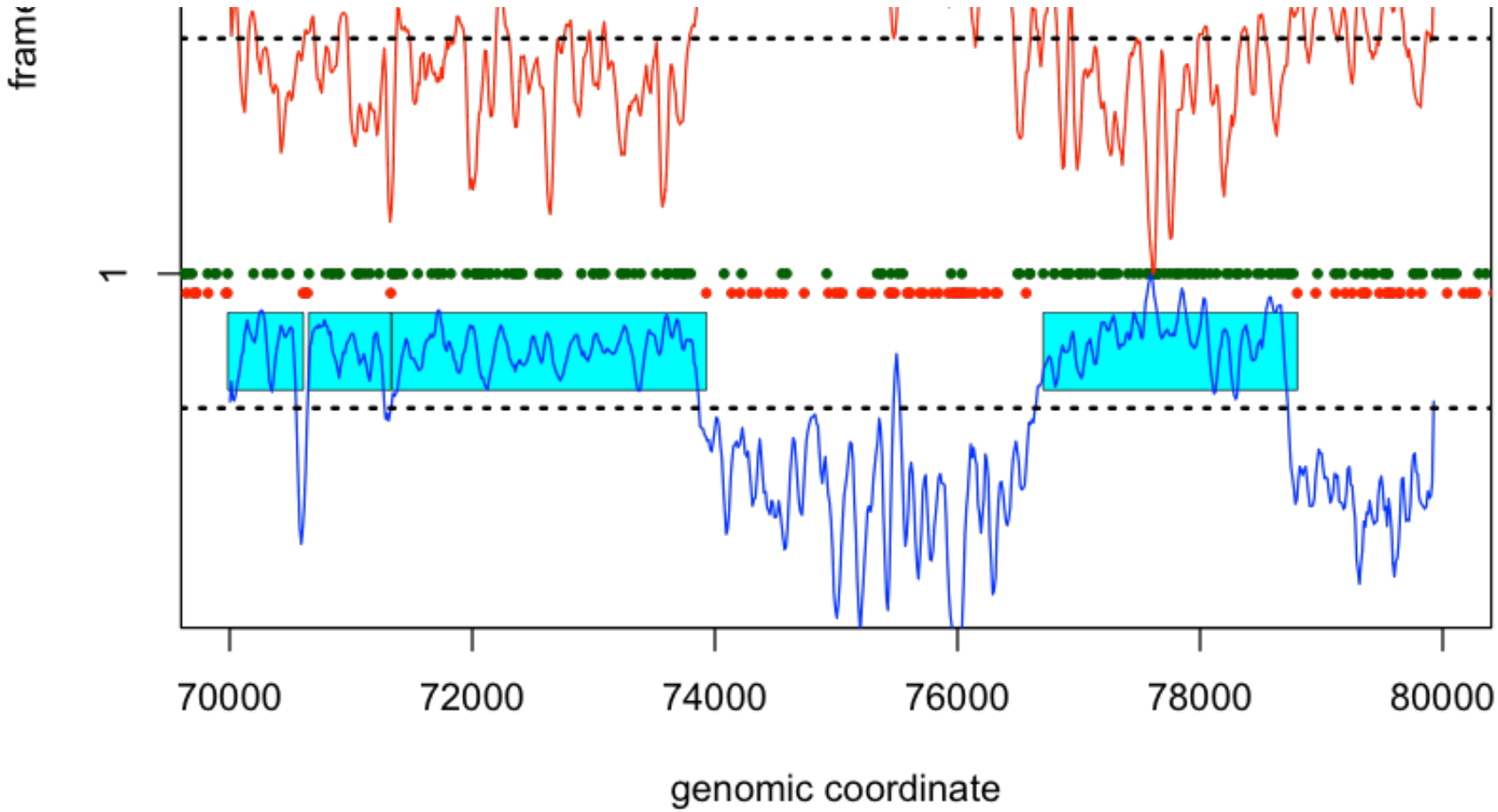


There is a clear separation between the likely coding and the likely non coding regions. We can also see that directly when checking the annotation.

In [57]:

```
PlotPhaseAndAnnotation(70000, 80000, Startpos, Stoppos, ecannot)

points(phase1$pos,phase1$scalelogp , t = "l", col = "blue")
points(phase2$pos,phase2$scalelogp+1 , t = "l", col = "red")
points(phase3$pos,phase3$scalelogp+2 , t = "l", col = "green")

abline(h = c(mean(phase3$scalelogp+2),
             mean(phase2$scalelogp+1),
             mean(phase1$scalelogp)),
       lwd =2, lty = 3)
```





Annotation of the ORFs

1. Compute the log odds ratio on all the ORFs that were annotated in the first part and use it to decide if an ORF is a coding gene. Check now how is the length distribution of the genes.
2. Add an ORF annotation for the reverse strand (You can modify the function by using the coordinate -1, -2 and -3 on the y axis).